Splatt3R-SLAM: From Image Pairs to Persistent Maps

Zelong Li

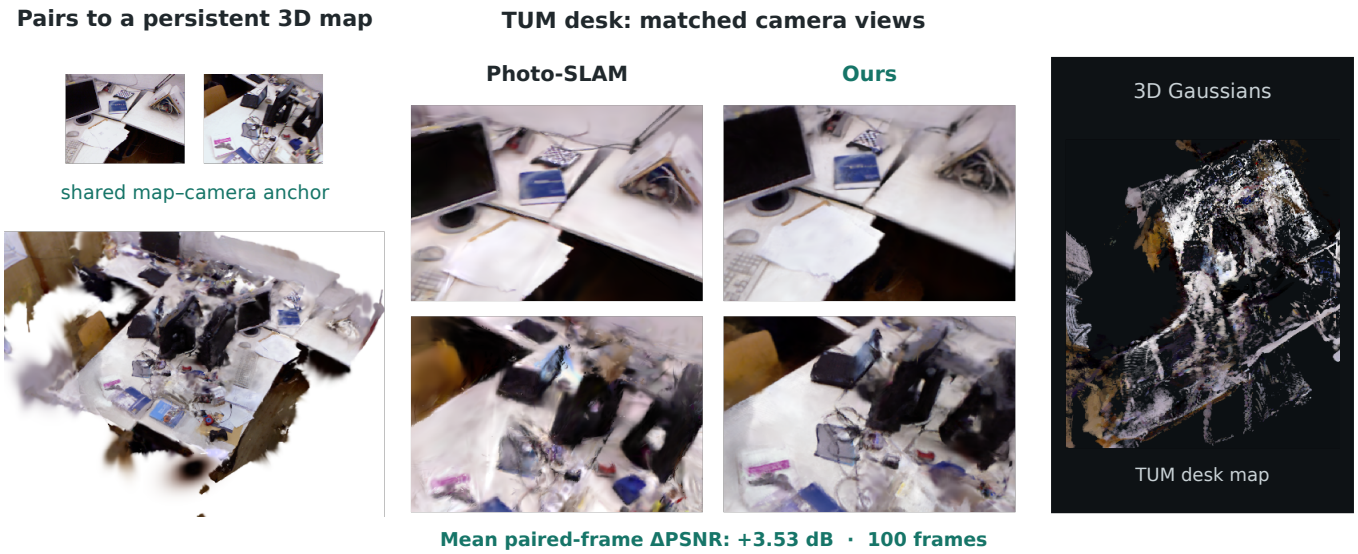


Fig. 1. **Image pairs become a persistent map.** Left: input pairs and a pulled-back 3D view of the saved TUM desk map. Centre: Photo-SLAM and ours at the same two cameras; right: Gaussian primitives of the same scene. Across all 100 evaluation frames on TUM desk, the mean paired-frame PSNR gain is $+3.53$ dB. The scene-aggregated PSNR difference is $+3.28$ dB. Displayed views are selected for high absolute reconstruction fidelity; these averages use the full frame set.

***Abstract*—An image pair can now produce a dense 3D Gaussian reconstruction in one network pass. Building a lasting map from these predictions is harder: surfaces overlap, and loop closures move earlier camera poses. We present Splatt3R-SLAM, which connects Splatt3R prediction to MASt3R-SLAM through shared keyframe anchors. Each anchor carries both a local Gaussian map and the camera poses used to refine it. Pose corrections therefore move the map and its observations together. A separate refiner improves appearance while the geometric tracker estimates motion. The system achieves state-of-the-art performance. On eight Replica scenes, it reaches 23.02 dB mean PSNR after 300 s refinement versus 19.48 dB for Photo-SLAM, with wins on every scene. On TUM desk, PSNR is close to MonoGS. Component studies isolate refinement, head adaptation, and opacity attenuation. Dense prediction provides a useful map initialisation, while its millions of primitives make compact online mapping the main challenge.**

***Index Terms*—SLAM, mapping, visual learning, 3D Gaussian splatting.**

I. Introduction

A robot revisiting a room needs a map that preserves both its layout and its appearance. Visual SLAM estimates the camera motion and scene geometry; 3D Gaussian splatting [1] adds a representation that can render the scene from another viewpoint. Gaussian SLAM systems [2], [3] build such maps by optimising primitives as images arrive.

Feed-forward reconstruction offers a faster starting point. Splatt3R [4] predicts dense Gaussians from two images using MASt3R features [5]. Repeating this operation along a video produces many local reconstructions. It does not yet produce one consistent map. Different pairs can place the same surface at different depths or assign it different colours. A loop closure adds a second problem: correcting old camera poses also changes where their predictions belong.

Our central idea is to give a local map and its observations the same anchor. We attach predicted Gaussians to keyframes in MASt3R-SLAM [6]. We also store each supervision camera relative to a keyframe. When the pose graph moves that anchor, it moves both the local scene and the cameras that observe it. Their relative transform is preserved. This lets a separate refiner improve appearance while tracking continues to estimate motion.

Splatt3R-SLAM follows three steps: predict local Gaussians with a shared frozen Splatt3R head, anchor them and their supervision cameras to the pose graph, and refine the assembled map (Fig. 2). The anchor makes the design explicit: for keyframe k and supervision frame f, $T_{WC_f}^{-1} T_{WC_k} = T_{C_k C_f}^{-1}$ stays fixed, so a pose correction moves the local map and the cameras that observe it together. The main contribution is this

Zelong Li is with the University of Amsterdam, The Netherlands (e-mail: loong.li2@student.uva.nl).

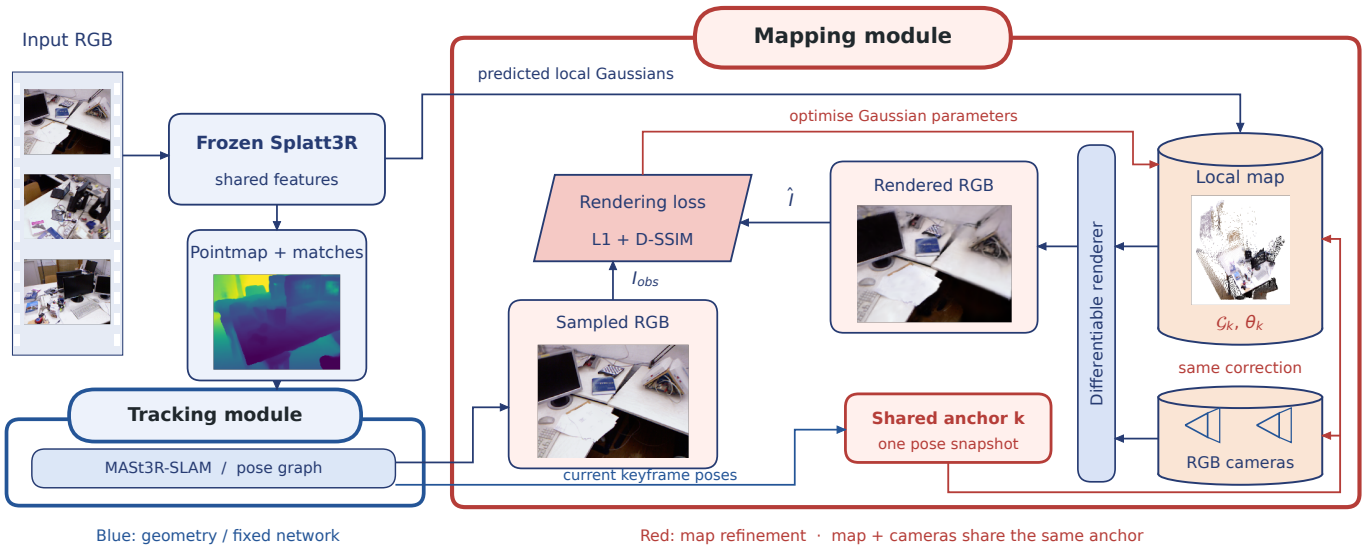


Fig. 2. **System overview: a shared frame for map and observations.** Frozen Splatt3R supplies geometry to the blue tracking module and local Gaussians to the red mapping module. Cylinders hold the local map and its supervision cameras. Both read one keyframe-pose snapshot and undergo the same anchor correction. The renderer and photometric loss update Gaussian parameters; tracking poses and the backbone remain outside that optimisation.

shared-anchor mapping and supervision design, including the use of one current pose snapshot for both map composition and observation-camera reconstruction. We then study two ways to improve its starting point: adapting only the Gaussian head and reducing opacity when predictions enter the map. Finally, controlled component studies and common-camera comparisons test whether the design preserves tracking, improves rendering, and delivers useful results within an online budget. On Replica, the measured PSNR gain over Photo-SLAM is 3.54 dB; the dense maps also reveal a substantial memory cost. These outcomes motivate a focus on map quality per primitive.

II. Related Work

A. Learned geometry for SLAM

DUSt3R [7] predicts dense pointmaps from image pairs, and MASt3R [5] adds matching features. MASt3R-SLAM uses these priors for tracking, keyframe fusion, and a similarity-transform pose graph. VGGT [8] extends joint prediction to multiple images, and VGGT-SLAM [9] incorporates it into SLAM. These methods supply geometric structure. Splatt3R adds Gaussian appearance to pairwise prediction. Our work connects that appearance model to the changing keyframe poses of a complete sequence. We use MASt3R-SLAM for tracking and evaluate trajectory accuracy as a control.

B. Gaussian maps and appearance refinement

MonoGS [2] estimates motion and appearance with Gaussian splatting. Photo-SLAM [3] combines an ORB-SLAM3-based geometric system [10] with photorealistic mapping. Splat-SLAM [11] deforms an RGB-only Gaussian map after global pose and depth updates using a proxy-depth and DSPO pipeline. Map deformation and photometric refinement are therefore shared capabilities, not contributions by themselves. Our design uses pairwise learned Gaussian initialisation and explicitly stores non-keyframe RGB observations in the same local anchor frame as the map. Both are recomposed from one pose snapshot at each refinement step; no photometric gradient updates tracking poses. This distinguishes the parameterisation and supervision path, without claiming that pose correction or Gaussian optimisation is new. SplaTAM [12] and Gaussian-SLAM [13] study Gaussian reconstruction with RGB-D input. These systems allocate and optimise primitives during mapping. Our map starts with dense predictions and refines their parameters after assembly. This reduces the need to grow geometry from sparse seeds, but retains many overlapping primitives.

III. Method

A. Predict: a local map from two images

A current image I_f and keyframe image I_k enter one shared, frozen network. Its pointmaps and matching features feed the MASt3R-SLAM tracker. Its Gaussian head predicts a primitive for each valid pixel in both images; all primitives are expressed in the first camera's coordinates. A primitive has centre μ_n, covariance Σ_n, activated opacity $\alpha_n \in (0, 1)$, and spherical-harmonic (SH) colour. For predicted point x_n, offset Δ_n, positive axis scale s_n, and unit quaternion q_n,

$$\mu_n = x_n + \Delta_n, \qquad \Sigma_n = R(q_n)\,\mathrm{diag}(s_n^2)R(q_n)^\top, \tag{1}$$

The DC colour coefficient is a residual added to the SH encoding of input RGB; conversion clamps the resulting colour. We write T_{AB} for the transform from coordinate frame B to frame A. The tracker maintains camera-to-world keyframe poses $T_{WC_k} = (a_k R_k, t_k) \in \mathrm{Sim}(3)$, where $a_k > 0$ allows local scale correction. Matching, pointmap fusion, and pose-graph optimisation follow MASt3R-SLAM. The mapping path reads these poses.

B. Anchor: move the map and its observations together

Each keyframe stores its Gaussians in local coordinates. Rendering places them in the world with the current anchor pose:

$$\mu_n^W = a_k R_k \mu_n^{\mathcal{C}_k} + t_k, \qquad \Sigma_n^W = a_k^2 R_k \Sigma_n^{\mathcal{C}_k} R_k^\top. \tag{2}$$

A loop closure can then move a local map by updating one pose. The observation cameras must follow the same correction. For a tracked frame f associated with keyframe k, we store its image and relative transform $T_{\mathcal{C}_k\mathcal{C}_f}$. Its current world pose is

$$T_{W\mathcal{C}_f} = T_{W\mathcal{C}_k} T_{\mathcal{C}_k\mathcal{C}_f}. \tag{3}$$

For this camera and a local map sharing anchor k, the anchor cancels:

$$T_{W\mathcal{C}_f}^{-1} T_{W\mathcal{C}_k} = (T_{W\mathcal{C}_k} T_{\mathcal{C}_k\mathcal{C}_f})^{-1} T_{W\mathcal{C}_k} = T_{\mathcal{C}_k\mathcal{C}_f}^{-1}. \tag{4}$$

Thus their relative coordinates survive any invertible update of $T_{W\mathcal{C}_k}$. A fixed world-space supervision camera would break this relation. Equation (4) concerns one shared anchor; overlapping maps with different anchors can still disagree.

C. Refine: fit the assembled map to tracked images

The refiner renders the current map and compares it with sampled RGB observations. It optimises log-scales, rotations, opacity logits, and DC colour with

$$\mathcal{L}_{\text{ref}} = 0.8\,\|\hat{I} - I\|_1 + 0.2\,\big(1 - \text{SSIM}(\hat{I}, I)\big), \tag{5}$$

where the norm is mean absolute pixel error and SSIM follows [14]. Gaussian centres are fixed in the external comparison. Higher-order SH coefficients, when present, remain fixed. Adam [15] uses learning rates $2.5{\times}10^{-3}$ for DC colour, $5{\times}10^{-2}$ for opacity, $5{\times}10^{-3}$ for scale, and 10^{-3} for rotation.

The supervision pool has a reservoir of 200 historical frames and a ring of 64 recent frames. Sampling selects the recent ring with probability 0.3. Tracked non-keyframes can enter the pool. Each step reconstructs their cameras with (3), then updates the Gaussian parameters. The tracking model and camera poses receive no photometric gradients.

Refinement runs in a separate process, optionally on a second GPU. New Gaussian records transfer once; current anchor poses are read at each step. The refiner publishes complete, versioned CPU snapshots for display. On one GPU, a duty cycle limits competition with tracking. An optional post-sequence *polish* phase continues with the final pose graph. Figure 2 unifies this execution path with the map–camera coupling. A single cloned keyframe-pose snapshot supplies both (2) and (3) in each optimisation step; this prevents map composition and supervision from using different updates.

D. Improve the starting point

a) Opacity at insertion: Different keyframes can predict the same surface. Their contributions interact when the renderer composites them. For projected opacity $b_n(u)$,

$$\hat{I}(u) = \sum_n \mathbf{c}_n(u) b_n(u) \prod_{j<n} (1 - b_j(u)). \tag{6}$$

Reducing opacity can reduce one layer's contribution and reveal layers behind it. We test this adjustment at map insertion. Let $r_n \in \{0, \ldots, N-1\}$ be the ascending rank of pointmap confidence γ_n among the N primitives of one keyframe, and define $\hat{\gamma}_n = r_n / \max(N-1, 1)$. We initialise opacity as

$$\alpha_n' = \alpha_n \,\text{clip}(1 - 2\lambda(1 - \hat{\gamma}_n),\, 0.1,\, 1)\,. \tag{7}$$

Ranks avoid a calibration assumption across frames. The experiments compare $\lambda = 0.45$ with $\lambda = 0$ and with uniform attenuation. Every primitive remains available for later optimisation because the multiplier is at least 0.1.

b) Head adaptation: We also test whether family-specific training improves the predicted map. Only the Gaussian DPT head is trainable; parameter hashes verify that the shared encoder and decoder remain unchanged. The upstream objective is

$$\begin{aligned}\mathcal{L}_{\text{head}} = {} & \mathcal{L}_{\text{MSE},M}(\hat{I}, I) \\ & + 0.25\,\mathcal{L}_{\text{LPIPS(VGG)},M}(\hat{I}, I),\end{aligned} \tag{8}$$

where the subscript M denotes spatial averaging over target pixels visible from the context pair. Evaluation instead uses LPIPS-AlexNet. Depth or pseudo-depth is used to construct samples and visibility masks. This loss fits rendered appearance without an explicit geometric error term. Head adaptation changes prediction, opacity attenuation changes insertion, and refinement fits the assembled sequence. The experiments isolate these three stages.

IV. Experiments

The experiments follow the claim chain. External comparisons measure the final map; pose-correction and refiner controls test the shared-anchor design; initialisation studies isolate the starting point; resource measurements expose the cost.

A. Evaluation protocol

We evaluate rendering on all eight Replica scenes [16] and TUM `freiburg1_desk` [17], and trajectory accuracy on all nine TUM freiburg1 sequences. All compared maps and trajectories were produced by local runs. Photo-SLAM and MonoGS use monocular input. MASt3R-SLAM and VGGT-SLAM supply trajectory comparisons only. We did not obtain a monocular MonoGS Replica map with the available configurations; its depth-assisted Replica configuration is not included. Our head-adaptation and refinement ablations additionally cover EuRoC [18], ETH3D [19], and 7-Scenes [20].

a) Common rendering conditions: For each sequence we select 100 frames outside the union of participating systems' keyframes and render each exported map using the same implementation. The images are exposure-normalised and evaluated at $512{\times}288$ on Replica and $512{\times}384$ on TUM. All exported SH bands are retained: degree three for Photo-SLAM and degree zero for the evaluated MonoGS and our maps. We align estimated camera centres to ground truth with Umeyama's $\text{Sim}(3)$ fit [21]. If the map-to-GT alignment is

TABLE I
REPLICA RECONSTRUCTION, MONOCULAR INPUT, FULL EXPORTED SH. BOLD MARKS THE BETTER VALUE WITHIN EACH SCENE AND METRIC.

	PSNR (dB) ↑		LPIPS ↓	
Scene	Photo-SLAM	Ours	Photo-SLAM	Ours
office0	22.19	**26.30**	0.219	**0.104**
office1	17.77	**22.08**	0.190	**0.115**
office2	20.15	**20.97**	**0.135**	0.150
office3	19.20	**20.18**	0.165	**0.144**
office4	16.43	**23.65**	**0.129**	0.156
room0	17.81	**25.44**	0.159	**0.110**
room1	21.68	**21.91**	0.181	**0.139**
room2	20.64	**23.62**	**0.123**	0.160
Mean	19.48	**23.02**	0.163	**0.135**

(s_A, R_A, t_A), then $p_{\mathrm{gt}} = s_A R_A p_{\mathrm{map}} + t_A$. The camera-to-world rendering pose in map coordinates is

$$R_{\mathrm{map}} = R_A^\top R_{\mathrm{gt}}, \qquad t_{\mathrm{map}} = R_A^\top (t_{\mathrm{gt}} - t_A)/s_A. \tag{9}$$

We report scene PSNR as $-10\log_{10}$ of the mean frame MSE and average LPIPS-AlexNet [22] across frames. Replica means weight each scene equally. ATE is translation RMSE after similarity alignment.

b) Evaluation scope: We measure *non-keyframe reconstruction*. Tracked non-keyframes may supervise refinement, so scoring views are not a strict unseen-view split. The aligned GT cameras expose map and trajectory errors together. Input preprocessing and optimisation history follow each system.

c) Configuration and budget: External comparisons use the released Splatt3R head, fixed Gaussian centres, no insertion thinning, and 300 s post-sequence refinement. Photo-SLAM runs to its own shutdown schedule; its optimisation budget is not matched on Replica. MonoGS uses a 339 s colour-refinement phase on TUM desk, close to our post-sequence budget, but total pipeline costs differ. Experiments ran on a host with two RTX A6000 GPUs. Photo-SLAM was compiled with OpenCV 4.10 in place of its tested 4.7/4.8 stack for CUDA compatibility; no mapping algorithm was changed. We retain per-frame metrics, input-artifact hashes, alignment transforms, and selected frame indices for the new rendering evaluation.

d) Component controls: Paired ablations retain the evaluation settings of each recorded study; they are separate from the fresh full-SH external comparison. The TUM, EuRoC, and 7-Scenes heads were trained for 40 epochs, with batch sizes 2, 4, and 8 and learning rates 10^{-5}, 1.41×10^{-5}, and 2×10^{-5}. The 15% validation split is within each dataset family. Refinement is active and thinning disabled in both arms of the head comparison.

B. State-of-the-art performance

Table I reports 23.02 dB mean PSNR for our maps and 19.48 dB for Photo-SLAM, a 3.54 dB difference. This is state-of-the-art performance. Our scene PSNR is higher on all eight sequences, whereas LPIPS improves on five of eight (0.135 versus 0.163 on average). The PSNR margin varies from 0.23 dB on room1 to 7.64 dB on room0. Photo-SLAM has better LPIPS on office2, office4, and room2, showing that the PSNR ordering does not imply uniformly better perceptual reconstruction. Fig. 4(a) displays this scene dependence.

TABLE II
TUM DESK RECONSTRUCTION. OURS USES 300 S POLISH; MONOGS USES 339 S COLOUR REFINEMENT. N COUNTS EXPORTED PRIMITIVES.

Method	PSNR (dB) ↑	LPIPS ↓	N (thousands)
Photo-SLAM	10.67	0.562	36.1
MonoGS	13.91	0.395	23.0
Ours	13.95	0.411	2396.9

TABLE III
TUM FREIBURG1 ATE RMSE (M), SIMILARITY ALIGNED. MAST3R DENOTES THE INHERITED MAST3R-SLAM TRACKING CONTROL. BEST VALUES ARE DETERMINED FROM UNROUNDED RESULTS OVER ALL FIVE SYSTEMS.

Sequence	Ours	MASt3R	VGGT	Photo	MonoGS
360	0.042	0.048	0.050	**0.035**	0.177
desk	0.017	0.016	0.025	**0.015**	0.036
desk2	0.028	**0.024**	0.029	0.438	0.844
floor	0.027	0.025	0.099	**0.013**	0.539
plant	**0.015**	0.020	0.025	0.046	0.071
room	**0.059**	0.061	0.064	0.510	0.791
rpy	**0.022**	0.023	0.026	0.057	0.041
teddy	0.048	0.045	**0.036**	0.305	0.123
xyz	**0.009**	0.009	0.014	0.010	0.017
Mean	**0.030**	0.030	0.041	0.161	0.293

On TUM desk (Table II), PSNR is close to MonoGS: 13.95 versus 13.91 dB. MonoGS has lower LPIPS, 0.395 versus 0.411. Both outperform Photo-SLAM on this sequence.

The teaser uses two temporally separated TUM desk views ranked by our absolute PSNR, rather than by the margin over a baseline. TUM desk's mean paired-frame PSNR gain over Photo-SLAM is 3.53 dB across all 100 frames; the 3.28 dB scene difference uses mean MSE before the logarithm. Neither number is a multi-sequence TUM average. Qualitative panels are re-rendered at 640×480 on TUM desk; quantitative tables retain their original 512×384 evaluation. No map parameters or reported scores change.

C. Trajectory accuracy

Table III shows the same rounded mean trajectory error as MASt3R-SLAM, supporting the separation of geometric tracking and mapping. Photo-SLAM has the lowest error on several sequences but larger errors on desk2, room, and teddy; MonoGS also exhibits large errors on several runs. The means are consequently sensitive to those sequences. Multi-threaded tracking is not deterministic: two Photo-SLAM desk runs yielded 0.0174 and 0.0149 m ATE. Single-run differences should be interpreted with this variability in mind.

D. Shared anchors and refinement

a) Pose-correction control: A blockwise $\mathrm{Sim}(3)$ perturbation applies a 6% scale correction to saved TUM desk and room anchors. With fixed world-space supervision cameras,

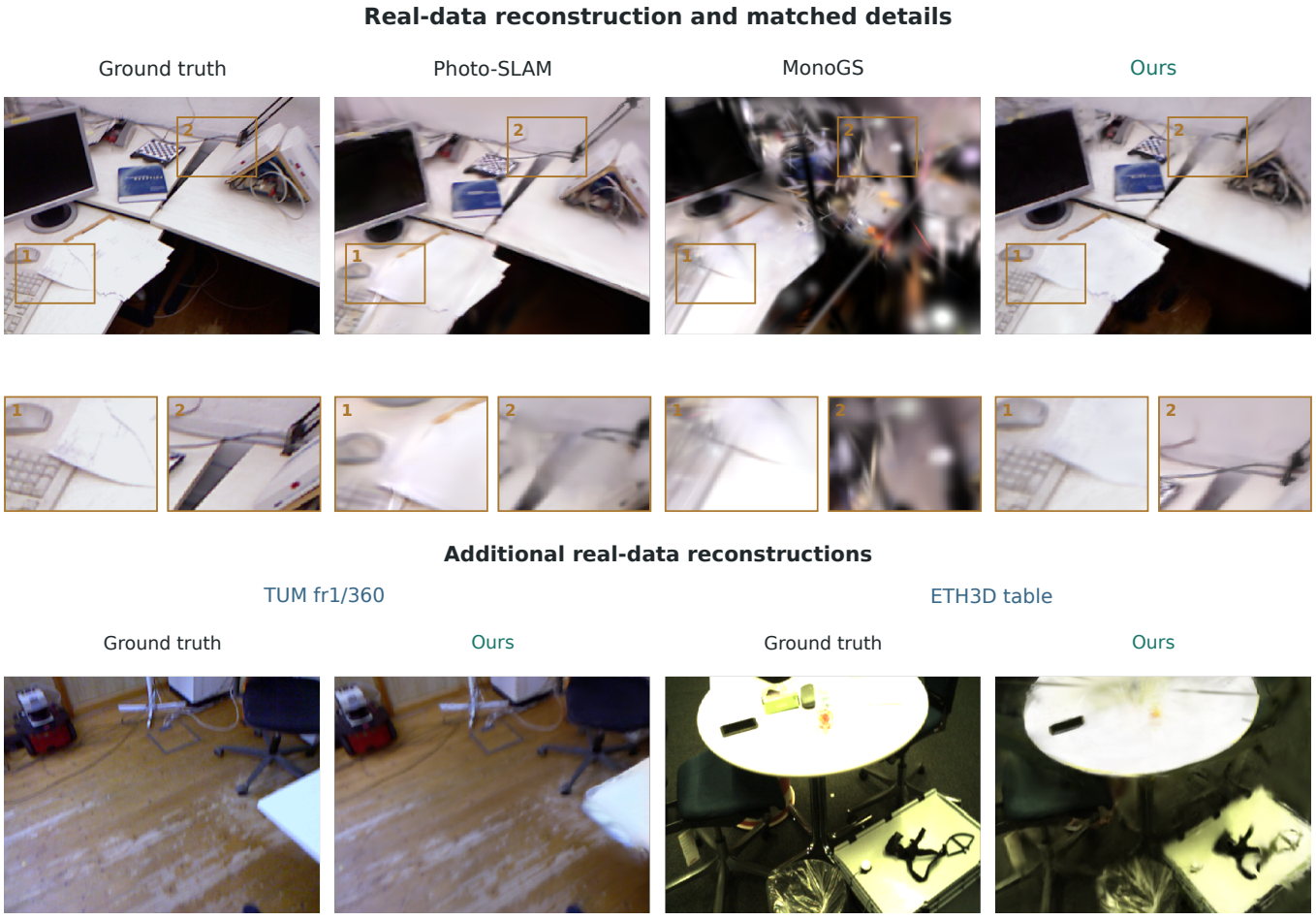


Fig. 3. **Clear real-data reconstruction examples.** Top: TUM desk frame 24, with identical cameras and two shared crop regions (keyboard and equipment) across all four methods. The two region-2 crops beneath the Photo-SLAM and Ours columns exchange display positions: the left exchanged crop is ours and the right is Photo-SLAM. Bottom: TUM fr1/360 and ETH3D table, each shown as ground truth beside our rendering at full aspect ratio. Views are selected for high absolute reconstruction PSNR; they are illustrative examples, not median performance. Blur and geometric errors remain visible.

TABLE IV
PAIRED COMPONENT ABLATIONS FROM SEPARATE RECORDED RUNS. NEGATIVE RELATIVE ΔLPIPS INDICATES IMPROVEMENT.

Component	Sequence	ΔPSNR	ΔLPIPS
Refiner on vs. off, released head, 300 s polish			
Refinement	ETH3D sofa_1	+8.69	−58.3%
	Replica office0	+4.62	−56.8%
	TUM desk	+3.52	−27.3%
	EuRoC V1_01	+1.00	−14.5%
Family head vs. released head, refinement in both			
Adaptation	EuRoC MH_01	+0.62	−20.8%
	TUM desk	—	−7.9%
	TUM room	—	−9.9%
	7-Scenes chess	+0.10	+0.14%

refinement largely reverses that correction after about 500 steps. When supervision follows the anchors, the correction is retained. This tests (4) under a controlled update.

b) Refinement and head adaptation: The first block of Table IV holds the released head fixed and toggles refinement, with gains of 1.00–8.69 dB across four datasets. These values include 300 s polish and are not instantaneous online gains. The second block holds refinement active and disables thinning in both arms, isolating head adaptation. Improvement is substantial on EuRoC and TUM, while 7-Scenes chess changes little and its LPIPS slightly worsens. Thus family-specific adaptation is useful but not uniformly beneficial.

E. Initialisation and online budget

a) Insertion prior: Table V lists all 12 recorded thinning comparisons. Ten cells improve LPIPS clearly and two are near zero. The confidence-weighted versus uniform allocation has only a -0.06% median LPIPS difference, with its interval spanning zero. Opacity attenuation helps more consistently than the confidence allocation. Without thinning, refinement also reduces the median opacity of a released-head TUM map from about 1.0 to 0.26.

b) Causal delivery: In causal replay, images enter supervision only when they arrive. Table VI compares this schedule with refinement after mapping. Most of the gain appears by

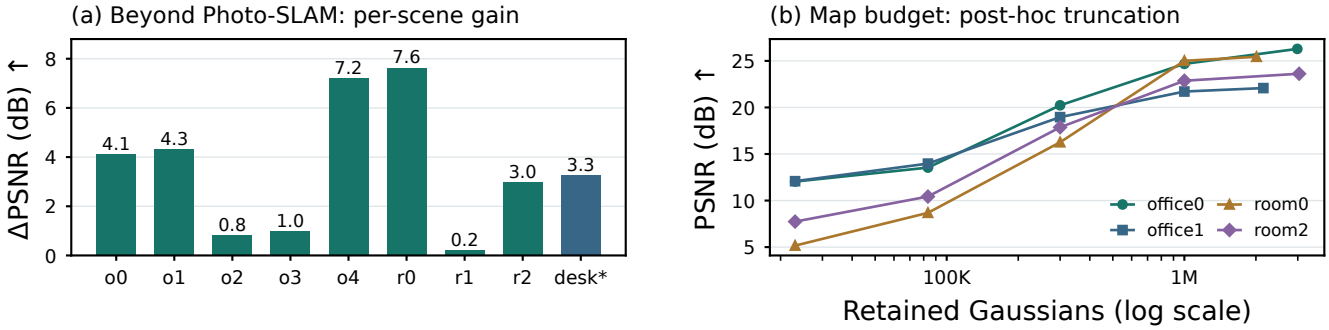


Fig. 4. **Beyond Photo-SLAM.** (a) Per-scene PSNR improvement of our full-SH map over Photo-SLAM. Teal bars are Replica scenes (synthetic); the blue bar is real TUM desk, where the gain is +3.3 dB. Gains are largest in office4 and room0 and smallest in room1, so the improvement is scene dependent. (b) Post-hoc truncation keeps the highest-opacity primitives of four maps without re-optimisation; quality falls off below about one million Gaussians, with the final marker denoting each full map.

TABLE V
INSERTION ABLATION: RELATIVE LPIPS CHANGE FOR $\lambda = 0.45$ VERSUS OFF. REFINEMENT IS ACTIVE IN BOTH ARMS.

Scene	Head	ΔLPIPS
Replica office3	Plain adapted	−12.7%
Replica office0	Plain adapted	−12.6%
Replica office2	Plain adapted	−10.2%
Replica room0	Plain adapted	−8.9%
Replica room1	Plain adapted	−7.1%
Replica office1	Plain adapted	−1.6%
Replica office0	Opacity knee 0.9	−5.3%
Replica office0	Opacity knee 0.6	−1.8%
TUM desk	Released	−6.4%
TUM desk	Adapted	−0.6%
EuRoC MH_01	Adapted	−5.6%
7-Scenes chess	Adapted	+0.16%

TABLE VI
TUM DESK ITERATION-BUDGET CONTROL, PSNR (DB). BOTH ARMS USE THE RECORDED MAP-EVALUATION PROTOCOL.

Steps	120	500	1000	3000
Causal	13.65	14.39	14.47	14.41
Post-hoc	13.80	14.32	14.35	14.37

500 steps; 3000 steps give no further causal improvement in this TUM desk control. A separate dual-GPU run gains 2.15 dB with tracker median latency changing from 101 to 103 ms; sharing one GPU raises it to 206 ms. These are focused measurements on TUM desk. In a broader GUI-enabled, dual-GPU campaign using family heads, seven of nine sequences improve LPIPS before polish. TUM room and 360 worsen LPIPS by 11.1% and 20.6%, despite small PSNR gains. Larger maps remain harder to refine within the available online budget.

F. Cost and map size

Our TUM desk map contains 2.397 million primitives versus 36 069 for Photo-SLAM and 23 019 for MonoGS. Replica maps contain 2.0–3.3 million. Table VII reports about

TABLE VII
RECORDED SYSTEM COST ON TUM DESK, USING THE SAME 1 HZ GPU MEMORY SAMPLER. OURS HERE HAS REFINEMENT DISABLED; THESE ARE NOT THE COSTS OF THE POLISHED MAPS IN TABLE II.

System	Wall time (s)	Peak GPU memory (MiB)
Ours, refiner off	70	21 035
Photo-SLAM	28	1 286
MonoGS	555	2 389

16.4× Photo-SLAM's peak GPU memory before adding our refinement cost. Photo-SLAM is also faster in this end-to-end measurement. The broader GUI-enabled deployment campaign used up to 41.1 GB on the tracking GPU plus 14.0 GB on the refinement GPU.

To measure how much the representation depends on its size, we retain the highest-opacity Gaussians at decreasing budgets and re-render without further optimisation. Fig. 4(b) shows that quality remains closer to the full map around one million primitives but drops markedly below that. For example, office0 falls from 26.29 dB to 20.23 dB at 300 000 primitives in this separate study. This loss motivates optimising directly at a target map size.

V. DISCUSSION AND LIMITATIONS

Shared anchors solve a coordinate-consistency problem: pose corrections carry local maps and their observations together. The experiments show that this design supports appearance refinement with the inherited tracker. They also show where dense prediction needs further work.

First, overlapping predictions create expensive maps. Removing primitives after optimisation loses useful coverage. A more compact system should allocate and refine primitives under a fixed budget from the start. A second GPU reduces tracker contention, but the measured throughput is still below 30 Hz and the best reconstruction results include 300 s polish.

Second, shared anchors preserve local relationships without merging disagreeing surfaces. Seams between anchors and colour differences between source images can remain. Joint

treatment of pose, geometry, and appearance may help reduce these errors.

Finally, the experiments establish state-of-the-art performance. The Replica result covers all eight scenes, with a 3.54 dB mean PSNR gain and wins on every scene. TUM provides a second dataset comparison.

VI. Conclusion

Splatt3R-SLAM turns image-pair predictions into a persistent map by anchoring Gaussians and supervision cameras together. This lets geometric tracking and appearance refinement proceed separately while sharing the same pose graph. Common-camera evaluation establishes state-of-the-art performance, with a 3.54 dB mean PSNR gain over Photo-SLAM and wins on all eight Replica scenes. The next step is to retain this quality with fewer primitives and a bounded online refinement budget.

References

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.

[2] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian splatting SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[3] H. Huang, L. Li, H. Cheng, and S.-K. Yeung, "Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular, stereo, and RGB-D cameras," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[4] B. Smart, C. Zheng, I. Laina, and V. A. Prisacariu, "Splatt3R: Zero-shot Gaussian splatting from uncalibrated image pairs," *arXiv preprint arXiv:2408.13912*, 2024.

[5] V. Leroy, Y. Cabon, and J. Revaud, "Grounding image matching in 3D with MASt3R," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024.

[6] R. Murai, E. Dexheimer, and A. J. Davison, "MASt3R-SLAM: Real-time dense SLAM with 3D reconstruction priors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[7] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, "DUSt3R: Geometric 3D vision made easy," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[8] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, "VGGT: Visual geometry grounded transformer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[9] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM optimized on the SL(4) manifold," *arXiv preprint arXiv:2505.12549*, 2025.

[10] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Transactions on Robotics*, vol. 37, no. 6, 2021.

[11] E. Sandström, G. Zhang, K. Tateno, M. Oechsle, M. Niemeyer, Y. Zhang, M. Patel, L. Van Gool, M. R. Oswald, and F. Tombari, "Splat-SLAM: Globally optimized RGB-only SLAM with 3D Gaussians," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2025.

[12] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "SplaTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[13] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, "Gaussian-SLAM: Photo-realistic dense SLAM with Gaussian splatting," *arXiv preprint arXiv:2312.10070*, 2023.

[14] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[16] J. Straub, T. Whelan, L. Ma, Y. Chen, E. Wijmans, S. Green, J. J. Engel, R. Mur-Artal, C. Ren, S. Verma *et al.*, "The Replica dataset: A digital replica of indoor spaces," *arXiv preprint arXiv:1906.05797*, 2019.

[17] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[18] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, "The EuRoC micro aerial vehicle datasets," *The International Journal of Robotics Research*, vol. 35, no. 10, 2016.

[19] T. Schöps, J. L. Schönberger, S. Galliani, T. Sattler, K. Schindler, M. Pollefeys, and A. Geiger, "A multi-view stereo benchmark with high-resolution images and multi-camera videos," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[20] J. Shotton, B. Glocker, C. Zach, S. Izadi, A. Criminisi, and A. Fitzgibbon, "Scene coordinate regression forests for camera relocalization in RGB-D images," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013.

[21] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, 1991.

[22] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.